# SCSIT, DAVV

Advanced Database Management Systems

## Assignment-I: E-R Diagrams

### Notation used in the diagrams

In this assignment I have drawn the diagrams in the classical Chen notation. A rectangle is an entity, an ellipse attached to it is an attribute, and a diamond on the line between two entities is a relationship. The number (1, M or N) written next to each line is the cardinality of that entity in the relationship.

| Symbol | Meaning |
|---|---|
| Rectangle | Entity (a thing about which data is stored) |
| Rectangle with double border | Weak entity (an entity that has no key of its own and depends on another entity) |
| Diamond | Relationship between two entities |
| Diamond with double border | Identifying relationship (connects a weak entity to its owner entity) |
| Ellipse | Attribute of an entity or of a relationship |
| Underlined attribute | Primary key of the entity |
| Dashed underlined attribute | Partial key of a weak entity (combines with the owner's key to form the full key) |
| 1, M, N on a line | Cardinality of that entity in the relationship (1 = one, M/N = many) |

# Case Study 1: Car Insurance Company

## 1.1 Problem statement

Construct an E-R diagram for a car-insurance company whose customers own one or more cars each. Each car has associated with it zero to any number of recorded accidents.

## 1.2 Understanding the problem

The question does not give any entity, attribute or key. So in this case everything (entities, attributes, primary keys, relationships and constraints) is assumed by me.

The company keeps records of its customers, the cars they own and the accidents of those cars. So the three main things (entities) are Customer, Car and Accident.

A customer must have at least one car, but a car can have no accident or many accidents. Also, one accident can involve more than one car (for example two cars hit each other). Because of this the relationship between Car and Accident is many-to-many. The damage amount is not the same for every car in an accident, so it cannot be kept in the Accident entity. It is stored as an attribute of the relationship (PARTICIPATED).

## 1.3 Entities and attributes (all assumed)

| Entity | Attributes (assumed) | Primary key |
|---|---|---|
| CUSTOMER | driver_id, name, address | driver_id |
| CAR | license_no, model, year | license_no |
| ACCIDENT | report_no, acc_date, location | report_no |

## 1.4 Relationships

| Relationship | Between | Mapping | Participation |
|---|---|---|---|
| owns | CUSTOMER - CAR | 1 : N | Customer: total (at least one car). Car: total (every car has an owner). |
| participated | CAR - ACCIDENT | M : N | Car: partial (may have no accident). Accident: total (at least one car). Attribute: damage_amount. |

## 1.5 Assumptions

- All entities, attributes and keys are assumed, because nothing is given in the question.
- A car is owned by only one customer, so CUSTOMER to CAR is one-to-many.
- Every customer owns at least one car, so participation of CUSTOMER in owns is total.
- A car can have zero accidents (partial participation), but an accident is recorded only if at least one car is involved.

- damage_amount belongs to the pair (car, accident), because each car gets a different damage in the same accident.

## 1.6 E-R diagram

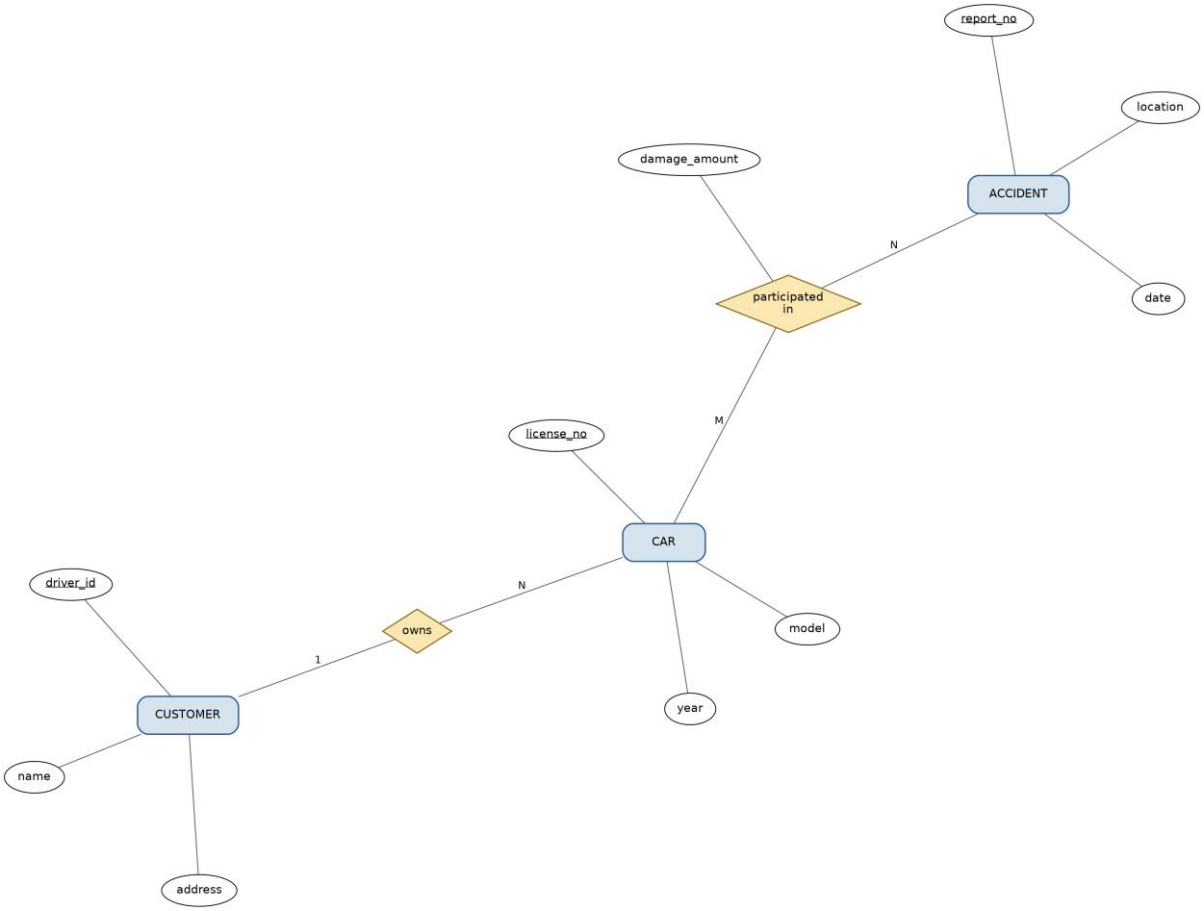



*Figure 1: E-R diagram for the car insurance company*

## 1.7 Relational schema (mapping to tables)

- **CUSTOMER**(driver_id, name, address)
- **CAR**(license_no, model, year, driver_id)
- **ACCIDENT**(report_no, acc_date, location)
- **PARTICIPATED**(license_no, report_no, damage_amount)

# Case Study 2: University Registrar's Office

## 2.1 Problem statement

A university registrar's office maintains data about courses (number, title, credits, syllabus and prerequisites), course offerings (course number, year, semester, section number, instructor(s), timings and classroom), students (student-id, name, program) and instructors (identification number, name, department, title). The enrolment of students in courses and the grades awarded to them must also be modelled. Construct an E-R diagram and document all assumptions about the mapping constraints.

## 2.2 Understanding the problem

The main point here is the difference between a course and a course offering. A course is the general subject, for example "DBMS", and it has a title, credits and syllabus. A course offering is one particular run of that course, for example "DBMS, 2026, odd semester, section A". The offering has its own timing, classroom and instructor.

An offering does not have a key of its own. To identify it we need the course number along with year, semester and section number. So COURSE_OFFERING is a weak entity and COURSE is its owner (identifying) entity.

A student is enrolled in an offering (not in the course directly), because the grade is given for that particular semester. Therefore ENROLMENT connects STUDENT and COURSE_OFFERING and it carries the attribute grade. Prerequisite is a relationship of COURSE with itself, since a prerequisite is also a course.

In the question, prerequisites is written as an attribute of a course, and instructor(s) as an attribute of an offering. But both of them refer to other entities (another course and an instructor). So I have not kept them as attributes. I modelled them as the relationships has_prerequisite and teaches.

## 2.3 Entities and attributes (given in the question)

| Entity | Attributes (as given) | Key (assumed) |
|---|---|---|
| COURSE | course_no, title, credits, syllabus | course_no |
| COURSE_OFFERING (weak) | course_no, year, semester, section_no, timings, classroom | course_no + year + semester + section_no |
| STUDENT | student_id, name, program | student_id |
| INSTRUCTOR | instructor_id, name, department, title | instructor_id |

**Given in the question:** the four entities and their attributes.

**Assumed by me:** the primary keys, the relationships between the entities, the mapping constraints (cardinality and participation), and the ENROLMENT entity.

## 2.4 Relationships and mapping constraints

| Relationship | Between | Mapping | Participation |
|---|---|---|---|
| has_prerequisite | COURSE - COURSE (recursive) | M : N | Partial on both sides. A course may have no prerequisite. |
| is_offered_as | COURSE - COURSE_OFFERING | 1 : N | Course: partial (may have no offering yet). Offering: total (must belong to a course). Identifying relationship. |
| teaches | INSTRUCTOR - COURSE_OFFERING | M : N | Offering: total (at least one instructor). Instructor: partial. |
| enrols | STUDENT - COURSE_OFFERING (through ENROLMENT) | M : N | Student: partial. Offering: partial. Attribute: grade. |

## 2.5 Assumptions about the mapping constraints

1. One course can have many offerings, but one offering belongs to exactly one course. So the mapping is 1 : N and the participation of the offering is total.
2. COURSE_OFFERING is a weak entity. Its partial key is (year, semester, section_no) and the full key becomes course_no + year + semester + section_no.
3. The question says "instructor(s)", so I have assumed that an offering can be taught by more than one instructor and one instructor can teach many offerings. This makes teaches an M : N relationship.
4. A student can enrol in many offerings and one offering has many students. So it is M : N and it is converted into the ENROLMENT table.
5. grade is an attribute of the enrolment (student + offering) and not of the student or the offering. It can be NULL until the grade is awarded.
6. A course can have many prerequisites and it can also be the prerequisite of many other courses, so has_prerequisite is a recursive M : N relationship.
7. timings can have more than one value (different days and times). For simplicity I have taken it as one attribute. It could be made a multi-valued composite attribute (day, start_time, end_time).

## 2.6 E-R diagram

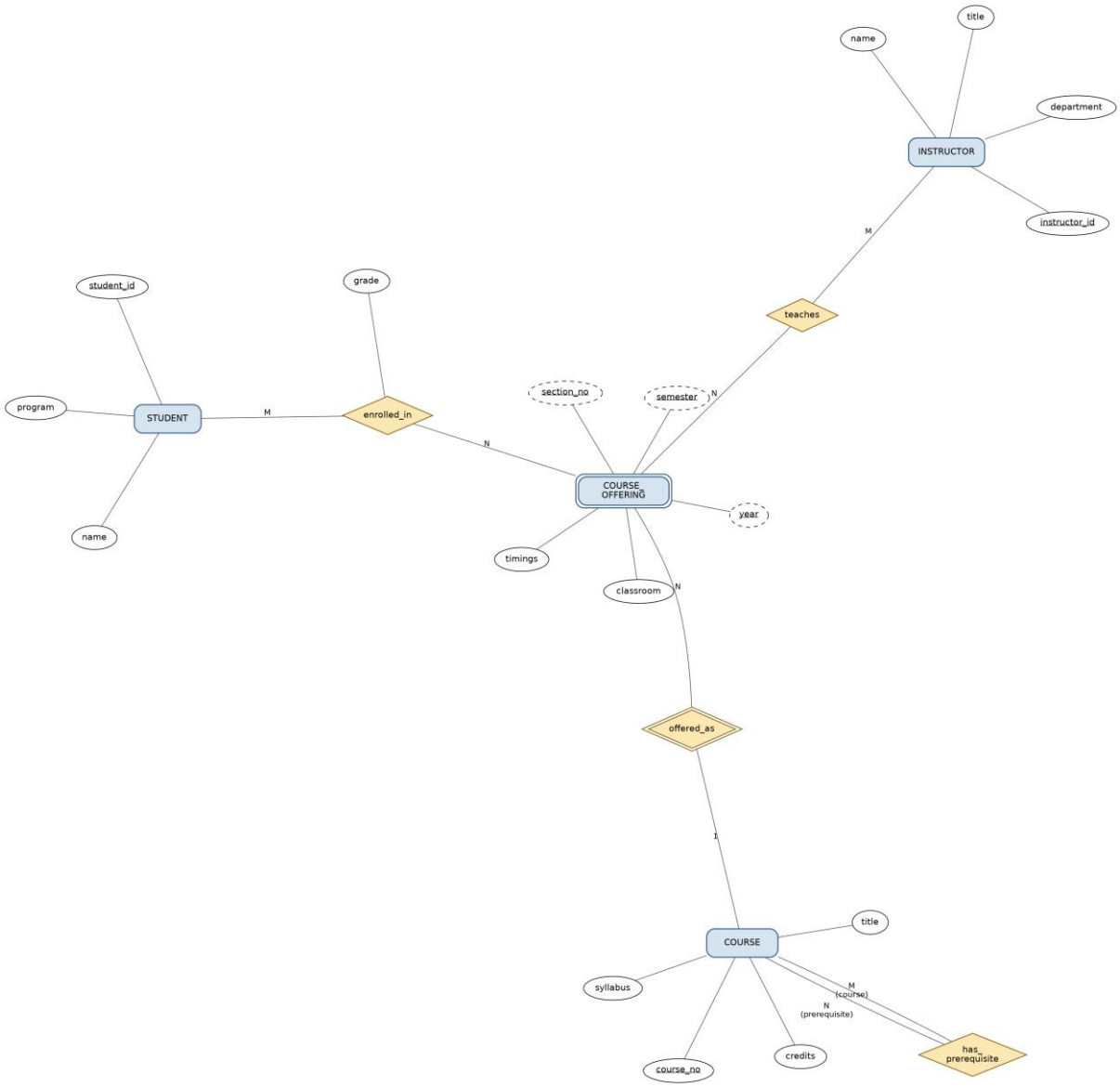



*Figure 2: E-R diagram for the registrar's office*

## 2.7 Relational schema (mapping to tables)

- **COURSE**(course_no, title, credits, syllabus)
- **PREREQUISITE**(course_no, prereq_course_no)
- **COURSE_OFFERING**(course_no, year, semester, section_no, timings, classroom)
- **TEACHES**(instructor_id, course_no, year, semester, section_no)
- **STUDENT**(student_id, name, program)
- **INSTRUCTOR**(instructor_id, name, department, title)
- **ENROLMENT**(student_id, course_no, year, semester, section_no, grade)

# Case Study 3: SCS Cricket League (SCL)

## 3.1 Problem statement

Requirements for a simple database for the SCS Cricket League: the SCL has many teams; each team has a name, a city, a coach, a captain and a set of players; each player belongs to only one team; each player has a name, a position (Batsman, Bowler or Wicket Keeper), a skill level and a set of injury records; a team captain is also a player; a game is played between two teams (host_team and guest_team) and has a date and a score. Construct a clean and concise E-R diagram.

## 3.2 Understanding the problem

In this case everything is given in the question (entities, attributes and relationships). So I have not added any new entity. I only added the identifiers needed for the keys.

The league has teams, the teams have players, and the teams play games against each other. So the entities are Team, Player, Game and Injury Record.

Since each player belongs to only one team, Team to Player is one-to-many. The captain is also a player, so I did not make a new entity for the captain. I used a one-to-one relationship "captained_by" between Team and Player. A player has a set of injury records, and an injury record cannot exist without its player, so INJURY_RECORD is a weak entity.

A game is played by two teams, host and guest. Both are teams, so there are two separate relationships between TEAM and GAME, host_team and guest_team.

## 3.3 Entities and attributes (given in the question)

| Entity | Attributes | Key |
|---|---|---|
| TEAM | team_name, city, coach | team_name |
| PLAYER | player_id, name, position, skill_level | player_id |
| INJURY_RECORD (weak) | injury_date, description | player_id + injury_date |
| GAME | game_id, game_date, score | game_id |

All attributes are as given in the question. The only additions are player_id and game_id (to have a key), and injury_date and description (the question only says "a set of injury records"). The coach and captain of a team and the set of players are shown as relationships.

## 3.4 Relationships

| Relationship | Between | Mapping | Participation |
|---|---|---|---|
| has | TEAM - PLAYER | 1 : N | Team: total (has players). Player: total (belongs to exactly one team). |

| Relationship | Between | Mapping | Participation |
|---|---|---|---|
| captained_by | TEAM - PLAYER | 1 : 1 | Team: total (exactly one captain). Player: partial (only some players are captains). |
| suffers | PLAYER - INJURY_RECORD | 1 : N | Player: partial (may have no injury). Injury record: total. Identifying relationship. |
| host_team | TEAM - GAME | 1 : N | Game: total (exactly one host team). Team: partial. |
| guest_team | TEAM - GAME | 1 : N | Game: total (exactly one guest team). Team: partial. |

### 3.5 Points taken from the question

- Each player belongs to only one team, so TEAM to PLAYER is 1 : N.
- A team captain is also a player, so I used a 1 : 1 relationship captained_by between TEAM and PLAYER and did not make a separate entity.
- A game is played between two teams (host_team and guest_team), so there are two relationships between TEAM and GAME.
- A player has a set of injury records, so INJURY_RECORD is a weak entity of PLAYER. If the player is deleted, his injury records are deleted too.

### 3.6 Extra constraints (not visible in the diagram)

- The captain must be a player of the same team.
- In one game host_team and guest_team must be different teams.
- score is kept as a single attribute. It can also be split into host_score and guest_score.

## 3.7 E-R diagram

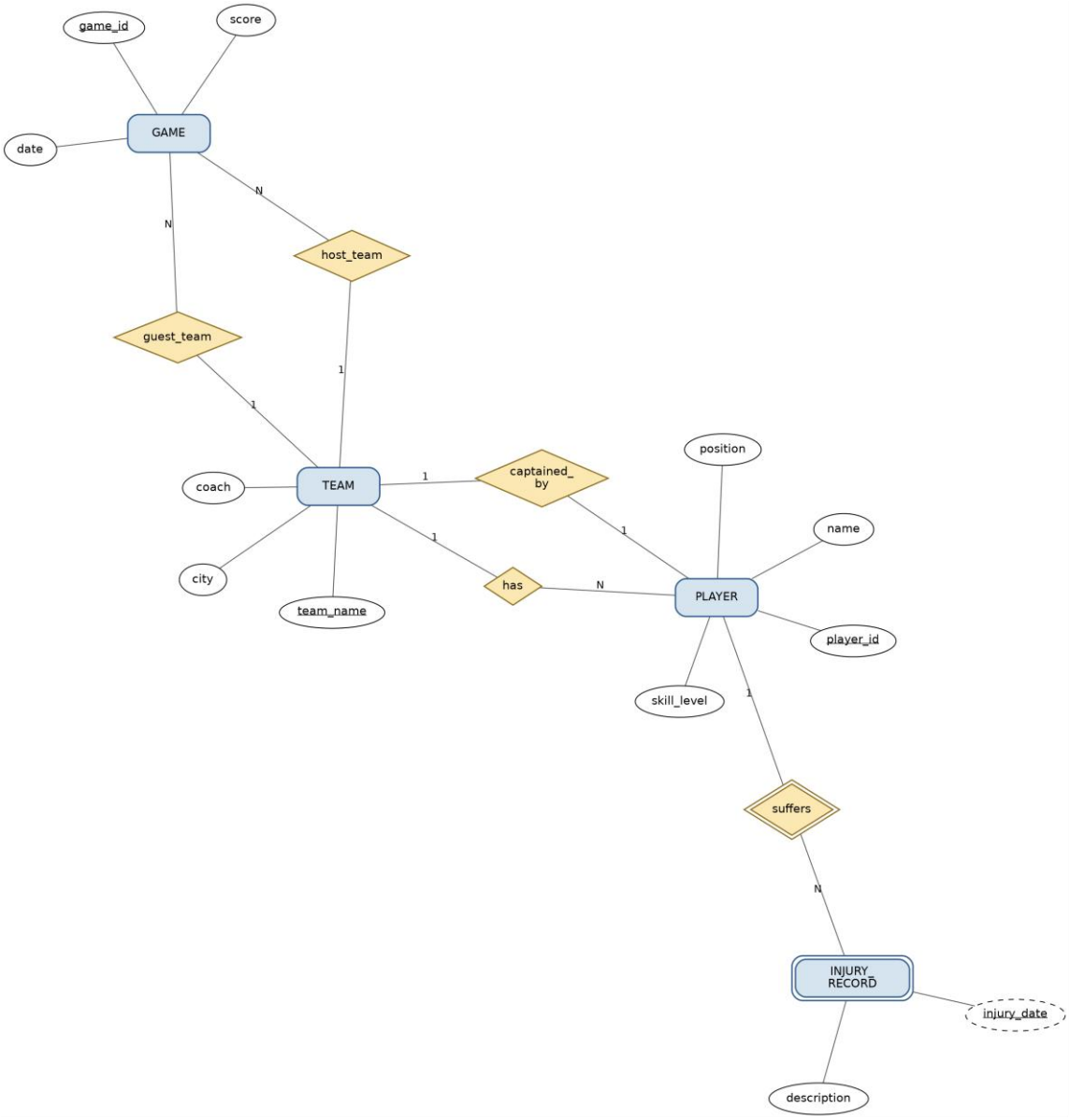



*Figure 3: E-R diagram for the SCS Cricket League*

## 3.8 Relational schema (mapping to tables)

- **TEAM**(team_name, city, coach, captain_id)
- **PLAYER**(player_id, name, position, skill_level, team_name)
- **INJURY_RECORD**(player_id, injury_date, description)
- **GAME**(game_id, game_date, score, host_team, guest_team)